

Models as a Service Breaking the GPU Silos on OpenShift AI 3.4

Table of Contents

Models-as-a-Service Introduction	1
Course Scenario - ChocoTech	4
1: MaaS architecture	6
2: MaaS Access Control & Governance	24
3. Model Deployment	38
4: Observability	58

Models-as-a-Service Introduction

Welcome to this **Red Hat OpenShift AI 3.4 - Models as a Service** QuickCourse! ---

The CIO at ChocoTech Industries just called an emergency meeting. "We've invested over \$500,000 in GPU infrastructure across four teams," she says, reviewing the quarterly report. "Yet our R&D team is waiting three weeks for capacity while Quality Control's GPUs sit idle 60% of the time. This doesn't make sense."

ChocoTech needs to align their AI Strategy to optimize for GPU utilization across business units, moving to a centralized, governed platform where all teams can access AI models efficiently.

The MaaS Mandate

Designed for Platform Engineers and Developers, this course provides the steps to build a governed Models-as-a-Service (MaaS) ecosystem on OpenShift AI 3.4. Rather than just showing you the "happy path," we will walk through the exact steps to build this architecture while intentionally navigating the common pitfalls, misconfigurations, and Day-2 troubleshooting realities you will face in production. You will learn how to immediately transform fragmented, isolated hardware into a fairly scheduled, multi-tenant shared service, empowering your business units to deploy agentic AI smoothly, securely, and at scale.



Experience Level: Intermediate

Prerequisites

To get the most out of this QuickCourse, you should come to the table with:

- Red Hat OpenShift administration experience (version 4.19.9 or later)
- Basic Kubernetes operator management knowledge
- Familiarity with core GPU concepts and generic GPU enablement
- An understanding of API concepts and REST principles
- Basic command-line and Python experience
- Networking fundamentals (Routing traffic via Gateway API)

What You'll Learn

In this hands-on QuickCourse, you will:

- **MaaS Strategy:** Understand how Models-as-a-Service solves complex GPU resource allocation and "Shadow AI" challenges.
- **Architect a Hub:** Design and implement a centralized model hub featuring governed, token-based consumption via Kuadrant and Authorino.
- **Configure and Deploy:** Set up MaaS using operators and deploy generative AI models leveraging the highly scalable `LLMInferenceService` architecture.

Enforce Governance: Implement strict token quotas and rate limits using `MaaSSubscription` and `MaaSAuthPolicy` custom resources.

- **Troubleshoot and Recover:** Diagnose `Degraded` Tenant resources, missing Gateway annotations, and 403/429 authorization failures.
- **Monitor and Report:** Enable telemetry, visualize API consumption, and export automated chargeback reports for financial accountability.
- **Integrate Seamlessly:** Connect AI models into downstream applications using OpenAI-compatible APIs with robust exponential backoff retry logic.

Required Operators and Components



MaaS requires the following operators and infrastructure components:

Component	Version	Installation Namespace	Status
Red Hat OpenShift	4.19.9+	N/A (Platform)	✓ Required
Red Hat OpenShift AI Operator	3.4+	redhat-ods-applications	✓ Required
Red Hat Connectivity Link Operator	1.2+	openshift-operators	✓ Required
Llama Stack Operator	3.4+	redhat-ods-applications	⚠ Required for dashboard UI
User Workload Monitoring	N/A	openshift-monitoring	✓ Required
PostgreSQL Database	14+	redhat-ods-applications	✓ Required
Gateway API	Built-in	openshift-ingress	✓ Required
Cluster Observability Operator	Latest	openshift-operators	○ Optional (Tech Preview)



Legend:

- ✓ Required for MaaS core functionality
- ⚠ Required for specific features
- Optional for enhanced capabilities

Estimated Time & Agenda

Plan for approximately **3 to 4 hours** to complete the full journey.

1. **Module 1: Architecture & Foundation** (45 mins)
2. **Module 2: Policies & Governance** (45 mins)
3. **Module 3: Model Deployment & Catalog** (60 mins)

Module 4: Observability, Chargeback, and Application Integration (50 mins)

5. Appendix A: Deep Dive - Gateway Routing & Troubleshooting (Reference)



Pacing Yourself: You can work through the modules at your own pace. We highly recommend **shutting down the expensive GPU lab environment** between active sessions!

Follow-up Resources

- [Red Hat OpenShift AI Documentation](#)
- [Red Hat Developer - OpenShift AI Resources](#)
- [Red Hat OpenShift AI 3.x Supported Configurations](#)
- [Red Hat OpenShift AI Product Page](#)

Authors and Contributors

Author

Karlos Knox
Content Services Architect

- **Last Updated:** May 2026
- **QuickCourse Version:** 3.4
- **Based on:** Red Hat OpenShift AI 3.4+ MaaS capabilities

Contact Information:

- **QuickCourse feedback:** Submit GitHub issues via the [QuickCourse repository](#).
- **Content updates:** Submit GitHub issues via the [QuickCourse repository](#).

Acknowledgments: This QuickCourse incorporates expert guidance from the Red Hat OpenShift AI business unit and product engineering architects working with enterprise AI deployments.

Special thanks to:

Taylor Jordan Smith,
James Harmison,
Anna Swicegood,
Caitlyn Rymarchyk,
Shumaila Yaseen.

Let's Get Started!

Ready to dive in? You'll start by meeting **ChocoTech Industries**—a rapidly growing chocolate manufacturer looking for a solution to their GPU silo crisis. Let's help them build a better AI infrastructure!

Course Scenario - ChocoTech

Your Role and Mission

Take on the role of a **Platform Engineer at ChocoTech Industries**, a modern chocolate manufacturing company. You have been assigned to solve a critical infrastructure challenge that is actively hindering multiple business units.

ChocoTech has teams across Quality Control, R&D, Marketing, and Supply Chain, all trying to leverage AI for their operations. However, each team has purchased and siloed their own GPU infrastructure, leading to a company-wide **"GPU Silo Crisis."**



- **Your Assignment:** Design and implement a Models as a Service (MaaS) platform on Red Hat OpenShift AI to transform fragmented GPU resources into a centralized, governed model hub.
- **The Objective:** Enable all lines of business to access generative AI models through a self-service catalog with proper governance. This will eliminate resource waste and accelerate business innovation.

The Current State

Four teams independently purchased and deployed their own GPU hardware:

- **Quality Control:** Running computer vision models for defect detection (2x A100 GPUs).
- **R&D:** Testing recipe optimization and flavor prediction models (4x L40S GPUs).
- **Marketing:** Analyzing customer sentiment and demand forecasting (2x L4 GPUs).
- **Supply Chain:** Running inventory prediction models (2x L40S GPUs).

Operational Pain Points

- **Severe Under-Utilization:** Quality Control's GPUs sit idle 60% of the time while R&D waits weeks for capacity, prompting unnecessary hardware purchases exceeding \$200,000.
- **Misallocated Expertise:** Data scientists spend 40% of their time debugging CUDA drivers, Docker containers, and Kubernetes manifests instead of tuning models and analyzing data.
- **Redundant Deployments:** All four teams independently downloaded and deployed the Llama-3 8B model, wasting 32GB of storage and compute capacity per deployment.
- **The "Shadow IT" Security Crisis:** Last week, the Marketing team got impatient and deployed a raw `LLMInferenceService` without any authentication. Without a centralized API gateway, there is zero visibility into who is sending what data to which model, making audit logs and PII filtering impossible to enforce.
- **No Chargeback Visibility:** Finance cannot justify infrastructure costs without clear visibility into model usage across users and applications.



The MaaS Opportunity We need to transition from hardware management to token delivery. MaaS transforms GPU infrastructure from dedicated, single-team clusters into a consumable backplane. Centralized IT manages the hardware, inference strategy, and governance policies (via KServe, Kuadrant, and the Gateway API), while the lines of business simply consume AI capabilities via tokens.

Your Vision of Success

If your MaaS implementation succeeds, here is the impact you will deliver to ChocoTech:

Immediate Improvements (Short-Term)

- **Eliminated Redundancy:** A single Llama-3 8B deployment serves all four teams, resulting in a 75% reduction in storage and compute waste.
- **Fair Resource Allocation:** Subscription-based quotas prevent Quality Control from monopolizing GPUs while R&D waits, ensuring balanced access.
- **Complete Audit Trail:** Every API request is logged with user identity, the model accessed, and tokens consumed to satisfy compliance requirements.
- **Simplified Developer Experience:** The Marketing team can integrate models via an OpenAI-compatible API in hours instead of weeks.

Strategic Benefits (Long-Term)

- **Accurate Chargebacks:** Token-based consumption tracking enables Finance to bill departments based on actual usage, justifying infrastructure investments.
- **Centralized Catalog Governance:** The platform team controls which models are available, preventing "Shadow AI" and standardizing security.
- **Maximized GPU Efficiency:** Intelligent routing and centralized inference reduce overall infrastructure costs by 40-50%.
- **Accelerated Innovation:** Developers focus on building applications instead of managing infrastructure, vastly increasing team productivity.

Common Questions

"Can we integrate MaaS with our existing AI models without major changes?"

Yes. Module 3 demonstrates deploying existing models from Hugging Face and custom model repositories using the llm-d and vLLM serving runtimes.

"What about security and compliance considerations?"

Module 4 covers governance policies, including rate limiting, subscription and authorization policy based access control, and audit logging through the Kuadrant policy layer.

"Is MaaS adoption sustainable for our team with limited Kubernetes expertise?"

Absolutely. This QuickCourse demonstrates operator-based configuration through OpenShift AI CRDs, minimizing low-level Kubernetes knowledge requirements while providing self-service capabilities via the dashboard.

1: MaaS architecture

The CIO at ChocoTech Industries just called an emergency meeting.

"We've invested over \$500,000 in GPU infrastructure across four teams," she says, reviewing the quarterly report. "Yet our R&D team is waiting three weeks for capacity while Quality Control's GPUs sit idle 60% of the time. This doesn't make sense."

ChocoTech needs to align their AI Strategy to optimize for GPU utilization across business units, moving to a centralized, governed platform where all teams can access AI models efficiently.

You've been tasked with implementing Models as a Service (MaaS) on Red Hat OpenShift AI to solve this challenge. In this module, you'll understand the business problem MaaS solves and explore the technical architecture that makes centralized model serving possible.

Learning objectives

By the end of this module, you'll be able to:

- Articulate the business and operational problems created by the GPU silo crisis.
- Explain how MaaS transforms GPU infrastructure from hardware management to token delivery.
- Map the three-layer MaaS architecture (KServe, Kuadrant, Gateway API) and their responsibilities.
- Verify that the foundational components are installed and healthy in your OpenShift cluster.
- Understand the complete request flow from API client through Gateway, policy enforcement, to model serving.

1.1: GPU silo crisis

ChocoTech Industries has four teams independently managing GPU infrastructure. Let's examine the operational pain points this creates.

The current state of GPU allocations across ChocoTech's business units:

- **Quality Control:** 2x NVIDIA A100 GPUs running defect detection models
- **R&D:** 4x NVIDIA L40S GPUs for recipe optimization
- **Marketing:** 2x NVIDIA L4 GPUs for sentiment analysis
- **Supply Chain:** 2x NVIDIA L40S GPUs for inventory prediction == The Solution: What is Models as a Service (MaaS)?

OpenShift Ai 3.4 - Models-as-a-Service (MaaS)

Before we dive into the technical architecture, let's define exactly what we are building to solve this crisis.

The Models-as-a-Service (MaaS) platform acts as a governance layer between users and the model serving infrastructure. This architecture enables centralized policy enforcement without modifying the underlying model serving components.

To achieve this, OpenShift AI 3.4 provides four core capabilities:

- **Subscription-based access control:** Define multiple subscriptions that grant specific groups access to models with configurable token limits. Users can belong to multiple subscriptions, with priority levels determining which subscription is used when multiple options are available.
- **Self-service API key management:** Users can create, list, and revoke their own API keys for model access. Administrators can also provision and manage API keys on behalf of users. API keys can be permanent or configured with custom expiration times, and individual keys can be revoked without affecting other keys for the same user.
- **Policy and quota management:** Enforce token limit policies to prevent resource exhaustion.
- **Usage tracking and observability:** Track consumption metrics for cost allocation, billing, and showback reporting to finance teams.

Out of Scope for this course

- **External models:** Route inference requests to models hosted by external cloud providers such as AWS Bedrock, Azure OpenAI, or Google Vertex AI through the same MaaS gateway used for locally deployed models. External models is a Technology Preview feature in Red Hat OpenShift AI.
- **Multi-runtime support:** Expose models served with llm-d or vLLM runtimes through MaaS governance. This provides runtime flexibility and enables consistent governance across different serving infrastructures. vLLM runtime support is a Technology Preview feature in Red Hat OpenShift AI.

The MaaS Impact: Problem vs. Payoff

How does this directly solve ChocoTech's "GPU Silo Crisis"? Review these core challenges and their corresponding MaaS solutions.

The Pain (The Problem)	The Payoff (The MaaS Solution)
GPU Under-Utilization Quality Control's \$120,000 A100 GPUs sit idle 67% of the time, while R&D waits weeks for capacity.	Dynamic Multi-Tenancy Hardware is centralized. MaaS allows dynamic accelerator slicing and multi-tenant sharing, driving up utilization and eliminating wait times.
Misallocated Expertise Data scientists spend 40% of their time debugging CUDA drivers and Kubernetes manifests.	Simplified Consumption Developers skip the infrastructure completely. They access powerful models instantly via secure, OpenAI-compatible API endpoints using self-provisioned API keys.
Redundant Deployments Four teams independently deploy the same Llama-3 8B model, wasting 128GB of storage and duplicate compute.	Centralized Catalog A single, highly optimized model deployment serves all business units simultaneously, drastically reducing storage and compute waste.
The Security "Black Box" Zero visibility into who is sending what data to which model.	Edge-Level Governance Centralized API gateways enforce strict authentication, zero-trust access, and token quotas directly at the network edge.
No Chargeback Visibility Finance cannot justify infrastructure costs because they cannot track which teams are consuming compute capacity.	Granular Token Tracking Exact consumption metrics (prompt/completion tokens) are tracked per user and subscription, enabling precise capacity forecasting and accurate departmental chargebacks.

Verify your understanding

Consider these questions based on ChocoTech's situation:

- How would centralized model serving help Quality Control and R&D share GPU capacity?
- What governance policies would prevent one team's script from monopolizing the shared resources?
- How does token-based consumption enable Finance to perform chargebacks?

1.2: MaaS architecture stack

Now that we understand the business problem—ChocoTech's fragmented hardware and lack of centralized governance—we must look at **how** OpenShift AI solves it.

Every request for model inference passes through three highly specialized layers: **The Gateway API**, **Kuadrant**, and **KServe**.

The Three-Layer Stack: Separation of Concerns

By decoupling the network routing, the policy governance, and the execution engine, MaaS provides extreme flexibility. You can adjust user rate limits without redeploying massive models, and you can swap serving runtimes without touching the security perimeter.

Layer	Core Component & Role	Enterprise Value ("The Why")
Layer 1: The Front Door	Gateway API (Traffic Routing) - Exposes a single, unified endpoint (<code>maas-default-gateway</code>) for all models across the cluster. - Handles TLS termination and path-based routing (e.g., <code>/v1/chat/completions</code>).	Simplified Connectivity: Developers do not need to manage complex, per-model DNS routing or insecure internal IPs. It provides a standard, OpenAI-compatible entry point.
Layer 2: The Bouncer	Kuadrant (Policy & Governance) Authorino: Validates tokens and enforces zero-trust authentication. Limitador: Tracks token consumption and enforces subscription-based token limits.	Security & Chargebacks: This layer mathematically guarantees that a team cannot exhaust GPU resources. It provides the exact token-usage telemetry required for Finance to bill departments accurately.
Layer 3: The Engine Room	KServe & vLLM/llm-d (Model Execution) - Orchestrates the actual model serving infrastructure. - Uses distributed inference frameworks (like llm-d) to spread large models across multiple GPUs.	Maximum ROI: By decoupling the request handling from low-level GPU tensor operations, KServe maximizes throughput and minimizes idle hardware time.

 MaaS Three-Layer Architecture showing Gateway API routing to Kuadrant for policy
Figure 1. MaaS Architecture: The Three Layers of Governed Model Serving

The Life of an Inference Request

To understand how these components work together, let's trace a real request from ChocoTech's Marketing team as they try to use the shared `llama-3-8b` model for a sentiment analysis application.

- 1. The Request Arrives (Gateway API):** The Marketing application sends a POST request to `/v1/chat/completions` containing their MaaS API key (passed as a Bearer token) and the prompt data.. The Gateway API catches this at the network edge.

Identity Verification (Kuardrant - Authorino): The Gateway pauses the request and asks Authorino, "Is this token valid, and what subscription level does this user have?" Authorino confirms the user is in the "Bronze" subscription group. Next it checks the Authorization policy which (TobeAdded) (If the token was invalid, it would immediately return a `401 Unauthorized` without touching the GPU).

3. **Policy Enforcement (Kuardrant - Limitador):** Policies (`MaaSAuthPolicy`) and subscriptions (`MaaSSubscription`) work together. Both use RBAC references (subjects or owners). A user must have both a matching policy (access) and a matching subscription (quota) to use a model. Limitador checks the Bronze subscription quota. "Has the Marketing team exceeded their 10,000 tokens-per-hour limit?" If they are within limits, the request is permitted. (If exceeded, it returns a `429 Too Many Requests`).
4. **Execution (KServe):** The validated, governed request is handed off to KServe. The vLLM runtime processes the prompt against the `llama-3-8b` weights and generates the response.
5. **The Audit Log:** The response flows back to the user, and Kuardrant meticulously logs the exact number of `prompt_tokens` and `completion_tokens` consumed by the Marketing application for the chargeback report.

This layered approach ensures that every request is authenticated, authorized, rate-limited, audited, and routed to the correct model serving infrastructure.

1.3: Enabling MaaS on RHOAI

You have explored the business problem and understand the three-layer architecture. Now it is time to activate MaaS on ChocoTech's OpenShift cluster. ChocoTech's GPU infrastructure is already operational from prior AI/ML experiments. Rather than starting from scratch, you will transform this fragmented hardware into a centralized MaaS platform by enabling the components that provide governance, routing, and multi-tenant model serving.

This module guides you through the exact activation sequence: verifying required operators, creating the infrastructure foundation, deploying the required database, and patching the OpenShift AI platform to expose the MaaS features.

Step 1: Configuration check & operator verification

Before applying any configurations, ensure your cluster meets the baseline requirements. You must be running OpenShift version 4.20.0 (officially 4.19.9) or later. Furthermore, specific operators and monitoring stacks must be installed. If any of these are missing, the deployment will silently fail.

Ensure the following operators and settings are active:

- **User Workload Monitoring:** This is absolutely critical for MaaS. Without it, the MaaS Tenant resource will show a `Degraded` status, and the platform will fail to collect token consumption metrics.
 - **Red Hat OpenShift Service Mesh 3:** Provides the underlying Istio networking that powers the Gateway API implementation.
 - **Red Hat Connectivity Link (v1.2+):** Delivers the Kuardrant governance framework.
-

Red Hat OpenShift AI (v3.4+): The core platform operator that manages the DataScienceCluster.

- **Leader Worker Set (LWS) Operator:** Mandatory for `llm-d` orchestration to manage models that exceed single-node VRAM capacity.
- **Cluster Observability Operator:** While MaaS core routing works without it, ChocoTech's Finance team needs this to view the MaaS observability dashboard (Tech Preview) for token consumption tracking and showback reporting.

Need a quick sanity check, click & run this:

If any operators are missing, install them from the OpenShift console (**Ecosystem** → **Software Catalog**) and wait for their status to reach `Succeeded` before continuing.

Step 2: The infrastructure foundation

The OpenShift AI operator acts as a strict gatekeeper. It will not provision the MaaS API unless the foundational network routing, security engines, and databases are actively running on the cluster.

Initialize the Governance Engine (Kuardrant)

The Connectivity Link operator manages Kuardrant (Authorino & Limitador), but Kuardrant itself must be explicitly initialized. A Kuardrant custom resource (CR) must be created and ready in the `kuardrant-system` namespace.

- Authorino: Validates tokens and enforces zero-trust authentication.
- Limitador: Tracks token consumption and enforces subscription-based token limits.

Create the namespace and Kuardrant instance:

```
cat <<EOF | oc apply -f -
apiVersion: v1
kind: Namespace
metadata:
  name: kuardrant-system
---
apiVersion: kuardrant.io/v1beta1
kind: Kuardrant
metadata:
  name: kuardrant
  namespace: kuardrant-system
spec: {}
EOF
```

Step 3: The Trap (Deploying without User Workload Monitoring)

Before we configure our Gateway and Database, let's look at the most common failure point when deploying MaaS.

In OpenShift AI 3.4, **User Workload Monitoring** is strictly required. This is the underlying mechanism that tracks token consumption and rate limiting across the platform. If it is disabled, the core MaaS

installation will fail to function properly and will display a `Degraded` status.

Let's simulate this common misconfiguration. We will skip enabling User Workload Monitoring for now, proceed with the installation, and watch how the platform reacts.

Expand for instructions to enable user Workload monitoring

Step 4: Create the MaaS Gateway

Next, we establish the Front Door. According to Red Hat prerequisites, a Gateway named `maas-default-gateway` must exist in the `openshift-ingress` namespace. **This becomes the single API endpoint for all of ChocoTech's model inference traffic.**

To ensure proper TLS hostname matching and external routing in a production environment, we will deploy the Gateway as an internal `ClusterIP` service and expose it using a native OpenShift Route with `reencrypt` TLS termination.

1. Create the infrastructure configuration

Create a ConfigMap to instruct the OpenShift service-ca-operator to auto-provision an internal TLS certificate (`maas-gw-service-tls`) and deploy the Gateway as a `ClusterIP` rather than an external load balancer.

```
cat <<EOF | oc apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: gw-options
  namespace: openshift-ingress
data:
  service: |
    metadata:
      annotations:
        # OpenShift service-ca-operator auto-provisions a TLS certificate into this Secret.
        service.beta.openshift.io/serving-cert-secret-name: "maas-gw-service-tls"
      spec:
        # ClusterIP avoids provisioning an external LoadBalancer; the OpenShift Route handles
        external ingress instead.
        type: ClusterIP
EOF
```

3. Deploy the Gateway

First, dynamically grab the OpenShift Gateway Controller class

```
export GW_CLASS=$(oc get gatewayclass -o custom-
columns=NAME:.metadata.name,CTRL:.spec.controllerName --no-headers | grep
"openshift.io/gateway-controller" | awk '{print $1}' | head -n 1)
```

Create the Gateway resource. Red Hat requires this resource to include two specific annotations for MaaS to function correctly. Notice that we omit explicit hostnames from the listeners to prevent SNI mismatches when the OpenShift Route passes traffic through.

```

cat <<EOF | oc apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: maas-default-gateway
  namespace: openshift-ingress
  annotations:
    security.opendatahub.io/authorino-tls-bootstrap: "true"
   .opendatahub.io/managed: "false"
  labels:
    app.kubernetes.io/name: maas-gateway
    istio.io/rev: openshift-gateway
    kuadrant.io/gateway: "true"
spec:
  gatewayClassName: ${GW_CLASS}
  infrastructure:
    parametersRef:
      group: ""
      kind: ConfigMap
      name: gw-options
  listeners:
  - name: http
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: All
  - name: https
    port: 443
    protocol: HTTPS
    tls:
      mode: Terminate
      certificateRefs:
      - group: ""
        kind: Secret
        name: maas-gw-service-tls
    allowedRoutes:
      namespaces:
        from: All
EOF

```

- ❶ **Authorino TLS Bootstrap:** Enables TLS communication between the Gateway and Authorino.
- ❷ **Managed Status:** Allows the MaaS controller to manage authorization policies without interference from the ODH Model Controller.

4. Expose the Gateway via OpenShift Route

Finally, create the OpenShift Route to act as the public entry point. This handles the external ingress and securely re-encrypts traffic back to the internal Gateway service.

```
cat <<EOF | oc apply -f -
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: openshift-ai-inference
  namespace: openshift-ingress
  annotations:
    router.openshift.io/service-ca-certificate: "true"
spec:
  port:
    targetPort: 443
  tls:
    insecureEdgeTerminationPolicy: Redirect
    termination: reencrypt
  to:
    kind: Service
    name: maas-default-gateway-`${GW_CLASS}`
    weight: 100
EOF
```

Understanding the Gateway API architecture in OpenShift AI

Red Hat OpenShift AI dictates the **requirements** of the Gateway API (specific namespace locations, Gateway names, and annotations) but leaves the physical proxy implementation up to the cluster administrator, noting that configuration templates are community-maintained.



The benefit of this architecture is the separation of infrastructure from application logic. As a platform engineer, you only build the Gateway. When ChocoTech deploys a model and checks the "Publish as MaaS" box, they do not need to write any networking rules. The MaaS controller automatically generates the `HTTPRoute` behind the scenes. That route dynamically attaches itself to the `maas-default-gateway`, securely linking the public endpoint through the Kuadrant governance layer directly to the internal model pod.

Optional: Verify GatewayClass Controller Compliance

While the exact infrastructure deployment of the Gateway is flexible, the official Red Hat documentation explicitly requires that the `GatewayClass` backing your MaaS deployment is configured to use the OpenShift Gateway Controller (`openshift.io/gateway-controller`).

Run this quick sanity check to verify that your `openshift-ai-inference` `GatewayClass` was built using the correct underlying controller.

```
oc get gatewayclass `${GW_CLASS}` -o jsonpath='{.spec.controllerName}'
```

Expected output:

If the output matches, your underlying Gateway API infrastructure is fully compliant with Red Hat's MaaS prerequisites.

Step 5: Configure Authorino Environment

The OpenShift service-ca-operator automatically handles the population of the cluster CA bundle. However, you are responsible for annotating the Gateway to trigger the automatic creation of the required EnvoyFilter for TLS. To get Models-as-a-Service (MaaS) running, you must configure the Authorino environment and verify the Gateway annotations.

First, you must configure the Authorino deployment with SSL environment variables so it can communicate securely with the MaaS API.

```
# Wait for the Authorino service to be created by the operator
echo "Waiting for Authorino service to initialize..."
until oc get service authorino-authorino-authorization -n kuadrant-system >/dev/null 2>&1; do
    sleep 5
done
echo "Authorino service is ready."

# Set SSL environment variables for outbound communication
oc -n kuadrant-system set env deployment/authorino \
SSL_CERT_FILE=/etc/ssl/certs/openshift-service-ca/service-ca-bundle.crt \
REQUESTS_CA_BUNDLE=/etc/ssl/certs/openshift-service-ca/service-ca-bundle.crt

# Generate the TLS certificate for Authorino
oc annotate service authorino-authorino-authorization -n kuadrant-system \
service.beta.openshift.io/serving-cert-secret-name=authorino-server-cert --overwrite

# Patch the Authorino resource to enable the TLS listener
oc patch authorino authorino -n kuadrant-system --type=merge --patch '
{
  "spec": {
    "listener": {
      "tls": {
        "enabled": true,
        "certSecretRef": {
          "name": "authorino-server-cert"
        }
      }
    }
  }
}'
```

Verify Authorino SSL Variables:

```
oc get deployment/authorino -n kuadrant-system -o
jsonpath='{.spec.template.spec.containers[0].env[?(@.name=="SSL_CERT_FILE")].value}'
```

Expected output:

```
/etc/ssl/certs/openshift-service-ca/service-ca-bundle.crt
```

Verify Gateway Annotation:

```
oc get gateway maas-default-gateway -n openshift-ingress -o
jsonpath='{.metadata.annotations.security\.opendatahub\.io/authorino-tls-bootstrap}'
```

Expected output: true

Note on Certificates: You must ensure your cluster already has a functional ingress controller with valid TLS certificates for external access before starting this procedure. If you are missing the `maas-db-config` secret or User Workload Monitoring, the Tenant resource will likely remain in a False or Degraded state.

Step 6: Deploy the MaaS API Database

In OpenShift AI 3.4, the platform features native API key management. To store these keys securely, you must have a PostgreSQL database and create a Secret named `maas-db-config` in the `redhat-ods-applications` namespace.

Deploy a lightweight PostgreSQL instance and the required secret for the MaaS API:

```

cat <<EOF | oc apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: maas-postgres
  namespace: redhat-ods-applications
spec:
  replicas: 1
  selector:
    matchLabels:
      app: maas-postgres
  template:
    metadata:
      labels:
        app: maas-postgres
    spec:
      containers:
        - name: postgres
          image: registry.redhat.io/rhel9/postgresql-15:latest
          env:
            - name: POSTGRESQL_USER
              value: "maasuser"
            - name: POSTGRESQL_PASSWORD
              value: "maaspass"
            - name: POSTGRESQL_DATABASE
              value: "maasdb"
          ports:
            - containerPort: 5432
---
apiVersion: v1
kind: Service
metadata:
  name: maas-postgres
  namespace: redhat-ods-applications
spec:
  selector:
    app: maas-postgres
  ports:
    - port: 5432
      targetPort: 5432
---
apiVersion: v1
kind: Secret
metadata:
  name: maas-db-config
  namespace: redhat-ods-applications
type: Opaque
stringData:
  DB_CONNECTION_URL: 'postgresql://maasuser:maaspass@maas-postgres:5432/maasdb?
sslmode=disable'
EOF

```

Step 7: Platform activation & The "Degraded" Reality

With the operators verified, Kuadrant running, the Gateway established, and the database online, you can now instruct the OpenShift AI operator to build the MaaS API layer.

Enable the MaaS Backend and Llama Stack

Apply the patch to enable MaaS and the Llama Stack Operator in the `DataScienceCluster` :

```
oc patch datasciencecluster default-dsc --type=merge -p '{
  "spec": {
    "components": {
      "llamastackoperator": {
        "managementState": "Managed"
      },
      "kserve": {
        "modelsAsService": {
          "managementState": "Managed"
        }
      }
    }
  }
}'
```

Diagnose the Failure

Now, check the status of the `Tenant` custom resource. Because we intentionally skipped enabling User Workload Monitoring, the OpenShift AI Operator will refuse to report the MaaS component as healthy.

```
oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o
jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
```

Expected Output: `False`

To see exactly **why** it failed, read the condition message:

```
oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o
jsonpath='{.status.conditions[?(@.type=="Ready")].message}'
```

The output will clearly indicate that User Workload Monitoring is missing. As a Platform Engineer, you will see this `False` or `Degraded` status frequently if prerequisites are missed.

The Fix: Enable User Workload Monitoring

Now that we have seen the failure, let's fix the cluster. Apply the following `ConfigMap` to enable User Workload Monitoring:

```
cat <<EOF | oc apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    enableUserWorkload: true
EOF
```

Wait a few moments, and check the `Tenant` status again:

```
oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o
jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
```

Expected Output: `True`

The `Tenant` resource is now healthy.

Enable the Dashboard (Gen AI Studio)

Even when deploying models via GitOps, administrators require the dashboard UI to manage user access tokens and monitor model usage. You must enable Gen AI Studio by patching the `OdhDashboardConfig` custom resource.

Apply the dashboard configuration patch:

```
oc patch odhdashboardconfig odh-dashboard-config -n redhat-ods-applications --type=merge -p '{
  "spec": {
    "dashboardConfig": {
      "modelAsService": true,
      "genAiStudio": true,
      "maasAuthPolicies": true
    }
  }
}'
```

What is the script below doing?

When you patch the `OdhDashboardConfig`, the OpenShift AI operator automatically spins up the Gen AI Studio components. However, you must explicitly tell the dashboard **how** to route traffic to the MaaS API. Without this script, the dashboard's internal proxy gets confused, loops back on itself, and crashes with an "Error loading components" message when you try to access the API Keys menu.

Here is exactly what this script accomplishes:

- **Prevents Race Conditions** (`oc wait`): Patching the dashboard config triggers the OpenShift AI operator to reconcile the deployment. This command pauses the script until the operator finishes its

work, ensuring our custom variables aren't overwritten mid-deployment.

- **Captures the Dynamic Route (export ROUTE_HOST):** Because we configured our Gateway Route to auto-generate a host based on the cluster's domain, we must dynamically query OpenShift to find out what that public URL actually is.
- **Injects the Routing Logic (oc set env):** This is the critical fix. It injects the MAAS_API_URL environment variable directly into the dashboard deployment. This explicitly forces the dashboard's Backend-For-Frontend (BFF) proxy to send all API Key and Model queries through our secure openshift-ai-inference Gateway, where Authorino can validate them.
- **Applies the Changes (oc rollout status):** Forces the deployment to spin up new pods with the injected URL and waits for them to become healthy before releasing the terminal.

```
# Wait for the dashboard deployment to be fully available after patching the config
oc wait --for=condition=Available deployment/rhods-dashboard -n redhat-ods-applications --
timeout=300s

# Capture the external Route URL
export ROUTE_HOST=$(oc get route openshift-ai-inference -n openshift-ingress -o
jsonpath='{.spec.host}')

# Inject the URL into the Dashboard
oc set env deployment/rhods-dashboard -n redhat-ods-applications
MAAS_API_URL=https://{ROUTE_HOST}/maas-api

# Wait for the Dashboard to restart with the new variable
oc rollout status deployment/rhods-dashboard -n redhat-ods-applications
```

Refresh your OpenShift AI browser window. The **Gen AI Studio - API Key generation tab** menu option will now appear in your left-hand navigation pane.

Step 8: Verification Summary

Before moving on to building ChocoTech's subscription plans in Module 2, run this comprehensive verification script to confirm all components deployed successfully.



Manual File Creation Required: You must create this file manually before adding the content (for example, by using `touch filename` or opening a new file directly in your text editor).

Copy/Paste Tip: Due to web terminal buffer limitations, pasting the entire block of text at once may result in nothing being pasted.

1. Copy and paste **tasks 1 through 5** first.
2. Once those are successfully pasted, copy and paste the **remaining text**.

```

cat > verify-maas-prereqs.sh <<'EOF'
#!/bin/bash

echo "=== MaaS Prerequisites Verification ==="
echo ""

# 1. OpenShift Version
echo "1. OpenShift Platform Version:"
OCP_VERSION=$(oc version -o json 2>/dev/null | jq -r '.openshiftVersion' 2>/dev/null || echo
"ERROR")
echo "   Version: ${OCP_VERSION}"
[[ "${OCP_VERSION}" == "ERROR" ]] && echo "   ❌ FAILED" || echo "   ✅ PASSED"
echo ""

# 2. OpenShift AI Operator
echo "2. Red Hat OpenShift AI Operator:"
RHOAI_CSV=$(oc get csv -n redhat-ods-applications 2>/dev/null | grep rhods-operator | awk
'{print $1}')
echo "   CSV: ${RHOAI_CSV:-NOT FOUND}"
[[ -z "${RHOAI_CSV}" ]] && echo "   ❌ FAILED" || echo "   ✅ PASSED"
echo ""

# 3. Red Hat Connectivity Link
echo "3. Red Hat Connectivity Link Operator:"
RCL_CSV=$(oc get csv -n openshift-operators 2>/dev/null | grep connectivity-link | awk '{print
$1}')
echo "   CSV: ${RCL_CSV:-NOT FOUND}"
KUADRANT_STATUS=$(oc get kuadrant kuadrant -n kuadrant-system -o
jsonpath='{.status.conditions[?(@.type=="Ready")].status}' 2>/dev/null || echo "ERROR")
echo "   Kuadrant Ready: ${KUADRANT_STATUS}"
[[ "${KUADRANT_STATUS}" == "True" ]] && echo "   ✅ PASSED" || echo "   ❌ FAILED"
echo ""

# 4. Llama Stack Operator
echo "4. Llama Stack Operator:"
LLAMA_STATE=$(oc get datasciencecluster default-dsc -o
jsonpath='{.spec.components.llamastackoperator.managementState}' 2>/dev/null || echo "ERROR")
echo "   Management State: ${LLAMA_STATE}"
[[ "${LLAMA_STATE}" == "Managed" ]] && echo "   ✅ PASSED" || echo "   ⚠️ WARNING: Required
for dashboard features"
echo ""

# 5. User Workload Monitoring
echo "5. User Workload Monitoring:"
UWM_ENABLED=$(oc get configmap cluster-monitoring-config -n openshift-monitoring -o
jsonpath='{.data.config.yaml}' 2>/dev/null | grep -o "enableUserWorkload: true" || echo "NOT
ENABLED")
echo "   Status: ${UWM_ENABLED}"
[[ "${UWM_ENABLED}" == "enableUserWorkload: true" ]] && echo "   ✅ PASSED" || echo "   ❌
FAILED"
echo ""

# 6. PostgreSQL Database Secret
echo "6. PostgreSQL Database Secret:"
DB_SECRET=$(oc get secret maas-db-config -n redhat-ods-applications -o
jsonpath='{.metadata.name}' 2>/dev/null || echo "NOT FOUND")
echo "   Secret: ${DB_SECRET}"
[[ "${DB_SECRET}" == "maas-db-config" ]] && echo "   ✅ PASSED" || echo "   ❌ FAILED"
echo ""

```

If the output shows  PASSED for the required components, ChocoTech's underlying infrastructure is officially primed for centralized governance. Proceed to the Summary and troubleshooting page to close out this module.

1.4: Summary & troubleshooting

You have successfully transformed ChocoTech's fragmented GPU infrastructure from a state of passive monitoring to active enablement. The physical and network foundations are now officially MaaS-capable.

Before we move on to establishing the actual governance rules, verify that your environment is healthy.



OpenShift AI 3.4 Requirement

In OpenShift AI 3.4, MaaS uses a subscription-based model for access control. If you are missing Custom Resource Definitions (like `MaaSSubscription`), check your Operator Lifecycle Manager (OLM) to ensure your cluster is subscribed to the 3.4 (or beta/EA) update channel, as version 3.3 does not support these features.

Day-2 Troubleshooting: What to check when things go wrong

As a Platform Engineer, you must know how to diagnose failures in the MaaS stack. If your verification checks failed in the previous step, use these methods to resolve them before proceeding to Module 2.

Issue: MaaS API pod is in `CrashLoopBackOff`

- **Check the Database Secret:** The `maas-api` pod strictly requires a database connection string. Check that the PostgreSQL database secret `maas-db-config` exists in the `redhat-ods-applications` namespace.
- **Verify Secret Contents:** Run `oc extract secret/maas-db-config -n redhat-ods-applications --to=-` to ensure the `DB_CONNECTION_URL` is formatted correctly.

Issue: Gen AI Studio is not visible in the dashboard

- **Verify Llama Stack:** The UI relies on the Llama Stack Operator. Run `oc get datasciencecluster default-dsc -o jsonpath='{.spec.components.llamastackoperator.managementState}'`. It must be `Managed`.
- **Verify Dashboard Patch:** Run `oc get odhdashboardconfig odh-dashboard-config -n redhat-ods-applications -o yaml | grep -A 10 dashboardConfig` to ensure `genAiStudio` is set to `true`.
- **Hard Reload:** Clear your browser cache and hard-refresh the page (Ctrl+Shift+R or Cmd+Shift+R).

Issue: Kuadrant pods are not starting

- **Check Connectivity Link operator logs:** Run `oc logs -n openshift-operators deployment/kuadrant-operator-controller-manager --tail=50`.
- **Verify Kuadrant CRDs are installed:** Run `oc get crd | grep kuadrant`.

Issue: Gateway status shows `PROGRAMMED=False` or `Unknown`

Verify Service Mesh: Ensure the control plane is healthy by running `oc get pods -n openshift-ingress`.

- **Check for Errors:** Run `oc describe gateway maas-default-gateway -n openshift-ingress`.
- **Root Cause:** The Service Mesh operator is not installed, or the Istio control plane is not deployed. Ensure the `openshift-ingress` namespace exists and contains running `istiod-` and `istio-ingressgateway-` pods. ---

Your journey progress

✅ Module 1

Architecture
Foundation

⌚ Module 2

& Policies & Governance

🔒 Module 3

Model Deployment

🚀 Module 4

Application Integration

“**Next steps:** You understand the business problem and have activated the MaaS platform. However, the foundation is operational but not yet configured for production. In Module 2, you will establish the actual governance rules, define subscription-based access controls (Bronze, Silver, Gold), and authorize Gateway traffic for your enterprise users.

2: MaaS Access Control & Governance

The CIO at ChocoTech Industries just called an emergency meeting. "The infrastructure is ready," she says, "but before we deploy any models, I need to know how we are going to control access and manage costs across business units. We cannot afford another Shadow IT incident."

She hands you the following requirements document:

Business Unit	Subscription Plan	Infrastructure Requirement
Quality Control	Gold Plan	Needs production-grade access with guaranteed availability and the highest token limits for real-time defect detection.
R&D	Silver Plan	Needs development access with moderate token limits for recipe optimization testing.
Marketing	Bronze Plan	Needs exploratory access with strict token limits for sentiment analysis to prevent budget overruns.

Furthermore, the CIO mandates that all access must be authenticated to create definitive audit trails, and strict token quotas must prevent any single team from monopolizing the GPU resources.

Learning objectives

By the end of this module, you will be able to:

- Explain how OpenShift AI 3.4's subscription-based governance replaces the legacy tier system.
- Create OpenShift groups and map them to logical business units.
- Deploy `MaaSSubscription` custom resources to define token limits and prioritize access.
- Deploy `MaaSAuthPolicy` custom resources to authorize Gateway traffic and avoid `403 Forbidden` traps.
- Generate secure API Keys and mitigate the "Group Snapshotting" security vulnerability. ---

The deployment strategy

To achieve this, you cannot simply deploy a model and hope it works. You must build the governance foundation from the bottom up. In this module, you will execute the following strategy:

1. **Organizational Structure:** First, you will create the native OpenShift groups for ChocoTech's business units (Quality Control, R&D, Marketing) and prepare a dedicated namespace.
2. **Subscription Governance:** Next, you will deploy `MaaSSubscription` resources to create the Gold, Silver, and Bronze plans, mapping them directly to your OpenShift groups.

Gateway Authorization: You will apply the `MaaSAuthPolicy` to physically open the network Gateway for your subscribed groups.

4. **API Key Generation:** Finally, you will generate the secure API keys that your developers and applications will use to interact with the models.

By the end of this module, ChocoTech's MaaS platform will be fully secured, authenticated, and ready to host high-performance models without the risk of resource exhaustion.

2.1: Subscription-based governance

Before we configure OpenShift groups or write any YAML, we must define the core mechanisms that make OpenShift AI 3.4 governance possible. In a traditional, decentralized AI setup, any user with access to an API endpoint can theoretically send massive, computationally expensive prompts to an LLM. This leads to immediate resource exhaustion and makes cross-departmental chargebacks impossible.

Models as a Service solves this by placing a strict, two-part governance layer between the user and the underlying KServe infrastructure.

The Strategic Separation of Access and Quota

In OpenShift AI 3.4, granting a team access to the AI platform requires exactly two custom resources working together. Think of it like joining a corporate gym:

1. **The `MaaSAuthPolicy` (The Keycard Reader / Access):** This is the physical enforcement mechanism at the door. It authorizes specific groups to access model endpoints through the API gateway.
2. **The `MaaSSubscription` (The Membership Contract / Quota):** This defines the terms once you are inside. It defines which models groups can access and configures the token rate limits for those models.



This intentional separation provides maximum flexibility. For example, administrators can create a generic "Premium" subscription that provides a massive token quota across 20 models, but use specific Authorization Policies to restrict a specialized team to only 5 of those models.

Design Recommendations & Best Practices

To properly govern the platform, ChocoTech's Platform Engineering team must adhere to the following architectural rules.

1. Recommendations for `MaaSSubscription` (Quota)

- **Centralized Deployment:** All subscriptions must be deployed into the central `models-as-a-service` namespace, rather than individual model namespaces, to allow platform-wide billing and quota management.
- **The Priority Numbering Scheme:** When a user belongs to multiple groups with different subscriptions, the system automatically selects the subscription with the highest priority level.
- **Use incremental gaps:** Number priorities with gaps (e.g., 10, 20, 30) rather than sequential numbers so you can easily insert intermediate priorities later.
-

Use **priority for defaults, not security**: Rely on priority levels for convenience, but use `MaaSAuthPolicy` to strictly enforce access control.

- **Handling Quota Exhaustion**: When developers hit the token limits defined in the subscription, they will receive an HTTP 429 Too Many Requests error. Developers should implement exponential backoff retry logic that reads the `Retry-After` response header to automatically pause and re-attempt the request.

2. Recommendations for MaaSAuthPolicy (Access)

- **The 403 Forbidden Trap**: Both a subscription and an authorization policy are strictly required. Without an authorization policy, users will receive 403 Forbidden errors even if they have a valid subscription.
- **The UI Synchronization Caveat**: If administrators create a subscription via the OpenShift AI Dashboard UI, an initial authorization policy is created automatically. However, changes made to the subscription later (such as adding or removing groups or models) require manual updates to the authorization policy to keep them synchronized.

3. Recommendations for MaaSModelRef (Registration)

- **Automatic Generation**: You generally do not need to create `MaaSModelRef` objects manually for internal models. Publishing as MaaS creates a `MaaSModelRef` object that notifies the MaaS system of the model's existence.
- **External Model Limits**: When routing inference requests to external cloud providers (like OpenAI or AWS Bedrock), token limits apply at two levels: the MaaS subscription level and the external provider level. Because all users share the same provider API key, the provider-level limit is shared across the entire group of users. Administrators must ensure the provider API key's limit is high enough to handle the aggregate consumption.

The ChocoTech Governance Strategy

To solve ChocoTech's GPU Silo Crisis, the CIO has mandated a custom governance mapping tailored to the company's business units.

In the upcoming sections, you will build the following structure using both `MaaSSubscription` and `MaaSAuthPolicy` resources:

- **Gold Plan (Priority 80)**: Mapped to the `chocotech-quality-control` group. This tier will have massive token limits to ensure the factory floor's defect detection models never experience throttling.
- **Silver Plan (Priority 50)**: Mapped to the `chocotech-rd` group. This tier will have moderate token limits for recipe optimization testing.
- **Bronze Plan (Priority 10)**: Mapped to the `chocotech-marketing` group. This tier will have strict token limits to prevent their exploratory sentiment analysis from wasting expensive GPU cycles.

Now that you understand the strategy and enforcement mechanics, proceed to the next section to begin building these OpenShift groups in the cluster.

2.2: Creating OpenShift groups

To enforce the CIO's subscription strategy, the MaaS platform needs a mathematical way to identify which request belongs to which team. We achieve this by utilizing native OpenShift Groups.

By mapping OpenShift groups to our MaaS Subscriptions, we gain massive enterprise advantages over standard public APIs:

- **Scalable Onboarding:** You add a new data scientist to a group, and they automatically inherit the correct API token limits and model access. No manual policy tweaking required.
- **Auditable Transparency:** Every API request is logged against the business unit (e.g., `chocotech-quality-control`), providing Finance with the exact metrics needed for departmental chargebacks.

Exercise 1: Create the business unit groups

Instead of creating individual users, let's create the three organizational groups that will soon be mapped to our Gold, Silver, and Bronze subscription plans.

Execute the following commands to create the ChocoTech groups:

```
oc adm groups new chocotech-quality-control
oc adm groups new chocotech-rd
oc adm groups new chocotech-marketing
```

Expected output: The terminal will confirm creation with three lines reading `group.user.openshift.io/<group-name> created.`

Next, to successfully complete the upcoming labs, you need to simulate being a member of the Quality Control team (our Gold Plan). Add your current user account to the QC group:

```
oc adm groups add-users chocotech-quality-control $(oc whoami)
```

Exercise 2: Prepare the models namespace

Create the new project for ChocoTech's centralized model hub:

```
oc new-project chocotech-models
```

Verify your foundation

Before moving on to the subscriptions, run this combined verification script to ensure your groups and labeled namespace are perfectly staged:

```
echo "=== Groups ===" && \
oc get groups | grep chocotech && \
echo "=== Gold Plan Membership ===" && \
oc get group chocotech-quality-control -o jsonpath='{.users[*]}'
```

If the output confirms your groups exist, your username is listed under the Quality Control group, and the `chocotech-models` namespace exist. Proceed to define the subscription limits.

2.3: Subscriptions and quotas

Exercise 1: Deploy the Subscriptions

We will use GitOps (Infrastructure-as-Code) to define the business logic for these three plans.



The "Pre-Wire" Trap: Notice the `modelRefs` section in the YAML below. Even though we will not deploy the `llama-3-8b` model until Module 3, MaaS allows us to "pre-wire" our governance.

Because the referenced model does not exist yet (missing the `MaaSModelRef` object), the OpenShift AI operator will immediately flag these subscriptions with a `Failed` phase status. This is expected! We are explicitly telling the platform: *"When a model reference named `llama-3-8b` appears in the `chocotech-models` namespace, apply these limits."*

Execute the following block to deploy all three `MaaSSubscription` policies into the centralized `models-as-a-service` namespace:

```

cat <<EOF | oc apply -f -
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSSubscription
metadata:
  name: gold-plan
  namespace: models-as-a-service
spec:
  priority: 80
  owner:
    groups:
      - name: chocotech-quality-control
    users: []
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
      tokenRateLimits:
        - limit: 50000
          window: 1m
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSSubscription
metadata:
  name: silver-plan
  namespace: models-as-a-service
spec:
  priority: 50
  owner:
    groups:
      - name: chocotech-rd
    users: []
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
      tokenRateLimits:
        - limit: 10000
          window: 1m
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSSubscription
metadata:
  name: bronze-plan
  namespace: models-as-a-service
spec:
  priority: 10
  owner:
    groups:
      - name: chocotech-marketing
    users: []
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
      tokenRateLimits:
        - limit: 2000
          window: 1m
EOF

```

Expected output: The terminal will confirm the creation of `gold-plan`, `silver-plan`, and `bronze-plan`.

Verification & Diagnosing the Failure

Verify that the MaaS operator successfully registered the subscriptions, and observe the expected failure state:

```
oc get maassubscription -n models-as-a-service
```

Expected output:

NAME	PHASE	PRIORITY	AGE
bronze-plan	Failed	10	8s
gold-plan	Failed	80	8s
silver-plan	Failed	50	8s

If a subscription shows a `Failed` phase status in production, your first troubleshooting step as a Platform Engineer is to check the conditions for the exact failure reason:

```
oc describe maassubscription gold-plan -n models-as-a-service
```

You will see an error indicating that the model reference cannot be found. This will automatically resolve in Module 3 once we deploy the model!

You have successfully defined **what** the groups are allowed to do. However, a subscription alone does not open the door. In the next section, we will deploy the `MaaSAuthPolicy` to physically authorize this traffic at the network edge.

2.4: Authorizing Gateway Access (MaaSAuthPolicy)

In the previous section, we defined the **quotas** using `MaaSSubscription`. However, as we learned in the Two-Part Access Rule, a subscription does not actually grant network access; it only dictates the speed limit once a user is inside.

To physically open the API Gateway and allow traffic to reach the models, we must deploy a `MaaSAuthPolicy`.

Exercise 1: Deploy the Authorization Policies

Just like the subscriptions, authorization policies must be deployed into the centralized `models-as-a-service` namespace.

We will create three policies to map our ChocoTech OpenShift groups (the `subjects`) to our target model (`llama-3-8b`).



Notice the `modelRefs` section in the YAML below. Even though we will not deploy the `llama-3-8b` model until Module 3, MaaS allows us to "pre-wire" our governance. Expect this policy to be in the failed state, until the model is deployed. We are explicitly telling the subscription: *"When a model reference named `llama-3-8b` appears in the `chocotech-models` namespace, apply these limits."*

Execute the following block to deploy the Auth Policies:

```
cat <<EOF | oc apply -f -
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: gold-access
  namespace: models-as-a-service
spec:
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
  subjects:
    groups:
      - name: chocotech-quality-control
    users: []
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: silver-access
  namespace: models-as-a-service
spec:
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
  subjects:
    groups:
      - name: chocotech-rd
    users: []
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: bronze-access
  namespace: models-as-a-service
spec:
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
  subjects:
    groups:
      - name: chocotech-marketing
    users: []
EOF
```

Expected output: The terminal will confirm the creation of `gold-access`, `silver-access`, and `bronze-access`.

Verification

Verify that the MaaS operator successfully registered the authorization policies:

```
oc get maasauthpolicy -n models-as-a-service
```

Expected output:

NAME	PHASE	AGE
bronze-access	Failed	9s
gold-access	Failed	9s
silver-access	Failed	9s

Check Tenant Readiness:

```
oc get tenant default-tenant -n models-as-a-service -o jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
```

Expected output: True

You have now successfully locked down the platform!

1. Only users in the QC, R&D, and Marketing groups can access the Gateway.
2. When they do, their traffic will be strictly throttled based on their Subscription tier.

The platform infrastructure is officially governed. The last step before we can hand this over to the developers is providing them with a secure way to log in.

2.5: Generating API Keys

With the organizational subscriptions defined, we must enable programmatic access for ChocoTech's applications. The Quality Control team's needs access to a Llama model to summarize test results.

OpenShift AI 3.4 introduces Native API Key Management, users can create, list, and revoke their own API keys for model access. Administrators can also provision and manage API keys on behalf of users. API keys can be permanent or configured with custom expiration times, and individual keys can be revoked without affecting other keys for the same user.

How API Keys Work in OpenShift AI 3.4

Before generating keys, it is crucial to understand how the MaaS controller handles identity and security at the network edge:

- **Show-Once Security:** API keys are cryptographically secure strings starting with `sk-oai-`. The plaintext key is only displayed once during creation and cannot be retrieved later.
-

Immutable Subscription Binding: Every API key is strictly bound to a single subscription at the time it is minted. If a user belongs to multiple groups and creates a key without specifying a subscription, the system uses the subscription with the highest priority level.

- **The Group Snapshotting Vulnerability:** API keys retain a snapshot of user group memberships from when the key was created. If a user is removed from a group, their existing API keys continue to grant access until you revoke them.

Self-Service vs. Programmatic Access

The platform supports two distinct workflows for generating keys:

1. **Creating API Keys by Users (Self-Service):** The platform empowers developers to manage their own access. For permanent keys, users navigate to **Gen AI studio** → **API keys**.
2. **Creating API Keys as Service Accounts (Programmatic Access):** For automated applications, platform administrators centrally provision API keys on behalf of users or applications.

Enforce Global Security Guardrails

By default, users can create "Permanent" keys. To enforce security policies, you can set a maximum expiration period for API keys across the entire platform.

Execute the following command to patch the centralized `Tenant` custom resource and enforce a strict 90-day maximum lifetime (`maxExpirationDays`) for all new API keys:



```
oc patch tenant default-tenant -n models-as-a-service \
  --type merge \
  -p '{"spec":{"apiKeys":{"maxExpirationDays":90}}}'
```

Expected behavior: Anyone attempting to create an API key with an expiration longer than 90 days will now receive an error.

Provisioning the Application Key (Service Account Pattern)

We need to generate a key for the Marketing team's automated sentiment analysis container. As a Platform Administrator, you will provision this key on their behalf.

1. In the OpenShift AI dashboard, navigate to **Gen AI studio** → **API keys**.
2. Click **Create API key**.
3. Fill out the creation form:
 - **Name:** `gold-admin` (Using a descriptive name is a best practice for tracking and simplifies revocation).
 - **Subscription:** Select the **Gold Plan** (This scopes the key's access to the models and token limits defined for Quality Control).
 - **Expiration:** Select `90 days`.

4.

Click **Create**.



Secure your key immediately! Save the API key in a secure location. Never commit API keys to version control. Store API keys in environment variables, Kubernetes secrets, or secret management systems such as HashiCorp Vault.

⚠ The Group Membership "Gotcha" (Shadow IT Simulation)

Imagine a scenario where a ChocoTech R&D engineer transfers to a different department, so IT removes them from the `chocotech-rd` OpenShift group.

Because API keys snapshot group membership at creation time, the engineer's old API key will **still work** and consume R&D's Silver Plan quota until the key reaches its 90-day expiration!

To immediately revoke access after a group change, administrators must manually revoke the API keys for that user. You must utilize the **Revoke user API keys** administrative action in the dashboard (Gen AI studio → API keys) to immediately cut off access when an employee leaves or changes departments.

Test the Gateway Authorization

You won't be able to complete this step, till a model is available, but including the steps here and will repeat them in the next section.

Now that the identity and permissions are established, let's test the authorization chain against the MaaS Gateway using our new key.

```
# 1. Export your newly created API key
export MAAS_API_KEY="sk-oai-your-copied-key-here"

# 2. Retrieve the MaaS Gateway URL
export CLUSTER_DOMAIN=$(oc get ingresses.config.openshift.io cluster -o
jsonpath='{.spec.domain}')
export MAAS_URL="https://maas.${CLUSTER_DOMAIN}"

# 3. Test authentication against the models endpoint
curl -s -w "\nHTTP Status: %{http_code}\n" \
  $MAAS_URL/v1/models \
  -H "Authorization: Bearer $MAAS_API_KEY" \
  -k
```

Decoding the Results

If your configuration is correct, you will receive the following output:

```
{
  "object": "list",
  "data": []
}
HTTP Status: 200
```

•

HTTP 200 OK: This is the critical success indicator. It proves that the Kuadrant gateway successfully validated your `sk-oai-` API key and mapped it to the correct Subscription and AuthPolicy.

- **Empty data array:** This is completely expected. We have successfully authenticated, but we have not published any models to MaaS yet.

ChocoTech's programmatic authentication is now secure and strictly governed. In **Module 3**, we will finally deploy the models and watch the MaaS catalog populate automatically!

2.6: Summary and troubleshooting

You have successfully transformed ChocoTech's OpenShift AI platform from a raw, ungoverned infrastructure into a highly secure, multi-tenant enterprise AI factory. If ChocoTech had opted for a generic public API, they would currently have zero internal cost control and no way to prioritize mission-critical workloads over experimental scripts. By building these policies directly into the OpenShift network edge, you have secured the company's ROI and established absolute Quality of Service (QoS).

What you accomplished

- **Organizational Identity:** Created OpenShift groups to represent specific business units (`chocotech-quality-control`, `chocotech-rd`, `chocotech-marketing`), ensuring audit trails are mapped to departments, not just individuals.
- **Programmatic Security:** You established a workflow where developers can self-serve secure, revocable `sk-oai-` API keys directly from the Gen AI Studio dashboard.
- **Subscription Definition:** Deployed `MaaSSubscription` custom resources into the `models-as-a-service` namespace, instructing the platform on exactly how to categorize ChocoTech's business units and enforce token quotas.
- **Edge-Level Governance:** Deployed `MaaSAuthPolicy` resources to physically authorize Gateway access for your groups. You successfully decoupled the inference request from the underlying GPU, ensuring that traffic limits are enforced mathematically at the network edge.

Knowledge check

Test your understanding of the subscription governance architecture before moving on:

1. Why is there no risk of "ungoverned access" if you deploy a model before creating its MaaS Subscription?

OpenShift AI operates on a default-deny security posture at the network edge. When you deploy a model and check "Publish as MaaS", the platform creates a `MaaSModelRef` object, but it does not grant any access. Users are strictly required to have both a subscription (to define token quotas) and an authorization policy (to grant API gateway access). Until an administrator explicitly links the model to a group via these policies, all incoming requests will be automatically rejected by Authorino with a `403 Forbidden` error.

2. What happens if a Bronze subscription user (Marketing) tries to exceed 2,000 tokens per minute?

Kuadrant's Limitador component, operating at the network edge, tracks the token consumption per subscription. It immediately rejects any requests exceeding the limit with an HTTP 429 Too Many Requests status code, preventing the request from ever reaching the KServe model pod. The user's application must wait until the next minute window resets.

3. How does the system know which subscription limits to apply to a programmatic user's request?

The API gateway intercepts the request and validates the provided API key (sk-oai-...). Because the key was permanently bound to the user's OpenShift group membership and highest priority subscription at the time it was created, the gateway immediately applies the correct token limits.

Day-2 Troubleshooting: Governance & Authentication

As you integrate applications, you will inevitably run into authorization errors. Use this cheat sheet to diagnose Gateway rejections:

Issue: Request returns 401 Unauthorized

- **Check API Key Prefix:** Ensure you're using the Authorization: Bearer <key> header format with the full API key including the sk-oai- prefix. OpenShift tokens are not valid for MaaS authentication.
- **Check Key Revocation:** The API key might be expired or manually revoked. Generate a new MaaS API key from the dashboard.

Issue: Request returns 403 Forbidden

- **The Missing AuthPolicy Trap:** A subscription grants quota, but an AuthPolicy grants access. Without an authorization policy, users receive 403 Forbidden errors even if they have a valid subscription.
- **Verify AuthPolicy Configuration:** Ask your administrator to verify that an authorization policy exists for your groups to access the model through the API gateway. Ensure your OpenShift group is explicitly listed in the subjects array.

Issue: Request returns 429 Too Many Requests

- **Token Limits Exhausted:** This means Kuadrant's Limitador component blocked the request because your subscription token limits are exhausted.
- **Implement Backoff:** Ensure your application's retry logic is respecting the Retry-After header value provided in the 429 response.

Your journey progress



Module 1

Architecture
Foundation



Module 2

& Policies & Governance



Module 3

Model Deployment



Module 4

Application Integration

“Next steps: Your security perimeter is established, but the catalog is currently

empty. In Module 3, you will deploy your first generative AI models (llama-3-8b and nemotron-8b). You will publish these models as MaaS endpoints (enabling the automatic generation of MaaSModelRef objects) and actively watch Kuadrant throttle traffic based on your Gold, Silver, and Bronze subscription limits.

3. Model Deployment

With the governance fences securely built at the network edge, it is time to bring ChocoTech's AI Model Catalog online. The R&D and Quality Control teams have submitted an urgent request: "We need access to Llama-3-8B for recipe optimization and defect analysis." For the second LLM you will substitute qwen3-4b for Nvidia Nemotron, and make Qwen3 model only available to the Marketing and R&D teams.

Your task is to deploy these generative AI models onto your GPU nodes, ensure they integrate with the MaaS Gateway you built in Module 1, and properly link them to the Gold, Silver, and Bronze subscriptions and authorizations you defined in Module 2.



For the lab environments with a single GPU, we'll swap models by stopping llama-3-8b model.

Learning objectives

By the end of this module, you will be able to:

- Provision model serving runtimes using `llm-d` (for distributed inference) or `vLLM`.
- Understand how the "Publish as MaaS" function automatically generates `MaaSModelRef` custom resources to register models with the gateway.
- Execute post-deployment governance steps to link newly deployed models to existing `MaaSSubscription` and `MASAuthPolicy` resources.

The deployment strategy

Deploying a model and making it accessible through MaaS is a strict, multi-step process. In this module, you will execute the following strategy to ensure complete governance:

Phase	Action
1. Hardware & Runtime Sizing	Analyze the hardware requirements for your target models and select the appropriate serving runtime (<code>llm-d</code> or <code>vLLM</code>).
2. The Deployment	Provision the model using the <code>LLMInferenceService</code> architecture. You will explicitly ensure the model does not bypass the MaaS gateway.
3. Registration (<code>MaaSModelRef</code>)	Use the "Publish as MaaS" feature to automatically generate a <code>MaaSModelRef</code> custom resource, notifying the MaaS platform that the model is ready for governance.
4. Post-Deployment Governance	A model published to MaaS is not immediately accessible. You will manually update your existing ChocoTech Subscriptions and AuthPolicies to append the newly generated model reference, officially closing the governance loop.

By the end of this module, ChocoTech's business units will finally have self-service, heavily governed access to state-of-the-art AI models.

3.1: Model Selection Strategy

ChocoTech's Platform Engineering team has received generative AI requests from three different departments. As the Models as a Service admin, you cannot simply provision GPUs and deploy models blindly. To prevent wasting expensive GPU resources, you must evaluate models based on their architecture and capabilities before deploying them.

Answering these questions will help:

- **What model architecture does the business actually need?**
- **Will this model physically fit on our GPUs?**
- **How do we ensure the deployment inherits our Models-as-a-Service (MaaS) governance rules?**

Before diving into the technical deployment steps, it is key to understand the "why" and "how" behind your configurations. The following content serves as a foundational introduction and reference guide for model selection, hardware sizing, and runtime architectures within OpenShift AI. It provides the necessary context to help you determine which models are the best candidates for your specific applications and infrastructure before you begin consuming cluster resources.

Evaluating model architecture: Dense vs. Sparse

Model architecture is not just a data science concern; it fundamentally impacts your GPU memory consumption, inference speed, and deployment complexity.

Architecture	Enterprise Characteristics
Dense Models	• Every single parameter in the neural network is active for every token generated. • This provides a highly predictable memory footprint and consistent inference latency, making capacity planning easier. • It offers a much simpler deployment architecture for platform teams. • Examples: Llama 2, Llama 3, Mistral 7B, Granite 3.0.
Sparse Models (Mixture of Experts)	• Activates only a specific subset of parameters per token using a complex routing mechanism. • Features a massively higher total parameter count, but requires less compute power per token. • Results in variable latency depending on which "expert" pathways the routing mechanism chooses. • Requires a highly complex deployment strategy involving expert-parallelism. • Examples: Mixtral 8x7B, DBRX, Grok-1.

The ChocoTech Decision: For the initial MaaS deployment, ChocoTech is standardizing on **dense models** (specifically Llama-3 8B). This minimizes deployment complexity and ensures predictable performance guarantees across all four departments.

Evaluating capabilities: Reasoning vs. Non-Reasoning

Beyond the underlying architecture, a model's capabilities determine its suitability for specific workloads and directly impact your computational costs.

Capability	Enterprise Characteristics
Reasoning Models	- Generates "invisible" intermediate reasoning steps before providing a final answer to the user. - Results in a significantly higher token output per query (reasoning tokens + final answer tokens). - Exhibits a slower time-to-first-token (TTFT) because the model "thinks" before it types. - Ideal for complex problem-solving, scientific analysis, and logic puzzles where accuracy outweighs speed.
Non-Reasoning Models	- Generates direct answers without explicit intermediate steps. - Results in lower token output per query and a much faster time-to-first-token (TTFT). - Highly cost-effective at scale for straightforward, conversational tasks.

The ChocoTech Department Mapping

Based on these technical assessments, you map the following model strategy for ChocoTech's business units:

Department	Architecture	Reasoning?	Rationale
Quality Control	Dense	Yes	Defect analysis requires multi-step reasoning and detailed explanations. The high demand for accuracy justifies the higher token cost.
R&D	Dense	Conditional	Recipe optimization heavily benefits from reasoning, but the team can fallback to standard generation for simpler tasks.
Marketing	Dense	No	Content generation is straightforward. Speed and cost-effectiveness matter significantly more to this team than deep reasoning.
Supply Chain	Dense	No	Their vendor chatbot requires ultra-fast responses for basic inquiries. Non-reasoning models are perfectly sufficient for FAQs.

The Hardware Math: Sizing the Model

Once a model is selected, you must ensure it physically fits on your infrastructure. A common mistake is assuming that a model's size correlates to standard disk storage, rather than Video RAM (VRAM).

As a rule of thumb, when running a dense Large Language Model at standard 16-bit precision (FP16 or BF16), **each parameter requires approximately 2 bytes of VRAM**. Let's calculate the footprint for Llama-3 8B:

- **Model Weights:** 8 Billion Parameters $\times$ 2 bytes = **~16 GB VRAM**.
- **KV Cache:** Generative AI requires active memory to store conversational context during inference; you must allocate an additional **2-4 GB VRAM** for this cache.
- **Total Required:** **~20-24 GB VRAM**.

This math proves that the Llama-3 8B model fits comfortably on a single NVIDIA A10 (24GB) or A100 GPU. However, this assumes that the context length will be reduced from its maximum limit to ensure it fits safely on the 24GB GPU.

Dealing with Massive Models (Quantization)

If ChocoTech wanted to deploy a massive 70B parameter model, it would require ~140GB of VRAM. To mitigate this without buying massive GPU clusters, platform engineers use **Quantization** (compressing the model's precision):

- **INT8 quantization:** Reduces memory consumption by approximately 50% with minimal quality degradation.
- **INT4 quantization:** Reduces memory consumption by approximately 75%, but comes with noticeable quality degradation, particularly for complex reasoning tasks.

The Serving Strategy: Why `llm-d` ?

To successfully govern a model in OpenShift AI 3.4, it **must** be deployed using the `LLMInferenceService` architecture. This architecture is specifically designed for large language models and integrates directly with the MaaS gateway you built in Module 1 to enforce subscription-based access control.

If you use the legacy deployment method (traditional `KServe InferenceService` resources with standard serving runtimes), you will completely bypass the MaaS governance features.

What is `llm-d` ?

`llm-d` (Large Language Model Distributed) is a specialized Kubernetes operator designed to orchestrate inference across multiple accelerators.

It utilizes the Leader Worker Set (LWS) Operator, which is absolutely mandatory for orchestrating models that exceed single-node VRAM capacity. Even if you are only deploying Llama-3 8B onto a single GPU today, using `llm-d` future-proofs your architecture. When ChocoTech eventually deploys that 70B

parameter model, `llm-d` seamlessly handles the tensor parallelism required to split that model across multiple GPUs using the LeaderWorkerSet architecture.



vLLM Runtime (Technology Preview)

MaaS supports models served with either `llm-d` distributed inference or `vLLM` runtimes. However, `vLLM` runtime support for Models-as-a-Service is currently a Technology Preview feature. If you plan to use it in your environment, you must explicitly enable the `spec.dashboardConfig.vLLMDeploymentOnMaaS: true` feature flag in the `OdhdashboardConfig` custom resource.

Now that we have verified the business requirements, calculated the hardware limits, and selected the appropriate runtime to guarantee governance, proceed to the next section to write the deployment manifest.

3.2: Lab - Deploying with Serving Runtimes



This section is under development

While advanced platform engineers often manage workloads via GitOps pipelines, OpenShift AI provides a cloud-native, wizard-driven user interface specifically designed for deploying models into a **Models-as-a-Service (MaaS)** fabric. Using the UI allows us to quickly configure inference runtimes, allocate dedicated GPU resources, and directly publish the model to the centralized MaaS gateway without writing complex Kubernetes Custom Resources by hand.

In this exercise, you will step into the role of the AI Platform Administrator and use the OpenShift AI model deployment wizard to deploy our target model (`llama-3-8b`) into the `chocotech-models` project.

Exercise 1: Create the ModelCar Data Connection

To simplify the model deployment process through the OpenShift AI dashboard, we will first create a URI data connection in the `chocotech-models` project. This configuration stores the exact OCI registry location of our ModelCar-packaged Llama 3 model and uses specific OpenDataHub annotations so the UI automatically recognizes it in the deployment dropdown menus.

Run the following command in your terminal to create the data connection secret:

```
cat <<EOF | oc apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: llama-3-8b
  namespace: chocotech-models
  labels:
    opendatahub.io/dashboard: "true"
  annotations:
    opendatahub.io/connection-type-protocol: uri
    opendatahub.io/connection-type-ref: uri-v1
    openshift.io/display-name: "Llama 8b ModelCar (v1.3)"
stringData:
  URI: oci://quay.io/redhat-ai-services/modelcar-catalog:llama-3.1-8b-instruct
type: Opaque
EOF
```

Exercise 2: Launching the Deployment Wizard

Now that the data connection is staged, let's deploy the model visually.

1. Open your **OpenShift AI Dashboard** and navigate to **AI Hub - Models** from the left-hand navigation menu.
2. From the project dropdown at the top of the page, ensure that **chocotech-models** is selected.
3. Click the **Deploy model** button to initiate the multi-step model deployment wizard.

Exercise 3: Navigating the Deployment Wizard

Follow the step-by-step instructions below to configure the model runtime, hardware allocation, and MaaS governance properties.

Step 1: Model Details and Location

Instead of manually typing out the OCI registry URL, we will leverage the internal data connection you just created.

1. For Model location, select the down arrow button then **Existing connection**.
2. Select **Llama 8b ModelCar (v1.3)**.: From the dropdown list, or choose the data connection we configured in the previous step (e.g., your pre-defined OCI Modelcar registry or internal storage path containing the model weights).
 - a. **Folder path**: If required by your data connection type, specify the subfolder path containing the weights.
3. **Model Type**: Select **Generative AI model (Example, LLM)***
4. Click **Next**.

Step 2: Model Deployment Configuration

1. **Name**: Enter `llama-3-8b`.
2. **Hardware profile**: Select **gpu-profile**.

3. **Deployment resource:** Select **manual**.
4. **Serving runtime:** From the dropdown menu, select ***Distributed inference with llm-d**.
5. **Number of replicas:** Leave this set to **1**. (In a production environment, you would scale this for high availability and usage requirements).
6. Click **Next**.

Step 3: Advanced Settings & Governance

This step is critical for integrating the model into ChocoTech's financial and security framework.

1. **Model Availability:** Check the box next to **Publish as MaaS**.

Checking **Publish as MaaS** automatically instructs OpenShift AI to generate a `MaaSModelRef` resource under the hood. This links the newly created model endpoint directly to the MaaS gateway, enabling our Authorino security policies and Kuadrant token quotas to intercept traffic.

1. **Configuration Parameters:** Select **Add custom runtime arguments**. This allows us to fine-tune the vLLM deployment. Add the following parameters:

```
--dtype=half
--max-model-len=20000
--gpu-memory-utilization=0.95
--enable-auto-tool-choice
--tool-call-parser=llama3_json
--chat-template=/opt/app-root/template/tool_chat_template_llama3.2_json.jinja
```

1. **Routing Settings:** Leave the Rolling update selection and model route timeout selections set to their defaults.
2. **Verify Subscriptions:** Ensure the model is mapped to the available tenant subscriptions (such as our Bronze Tier) so your developers can successfully authenticate against it using their assigned API keys.
3. Review your configuration choices on the summary pane and click **Deploy model**.

Exercise 4: Verifying the Deployment Status

Once submitted, the platform transitions the model status to `Progressing` while it pulls the weights and initializes the inference engine.

1. Navigate back to the **Model Serving** dashboard.
2. Under the `chocotech-models` project view, locate your `llama-3-8b` model deployment.
3. Wait until the status indicator shifts from a blue spinning icon to a green checkmark indicating **Loaded** or **Active**.

In the next section, we will step out of the administrator interface and into the developer's shoes to verify that our newly deployed, UI-managed model is actively accepting authorized API traffic!

3.3: Lab - Distributed Inference Configuration

If ChocoTech had deployed this model using standard open-source tools, the Platform Engineering team's job would not be finished. They would still need to manually write documentation, securely email the endpoint URL to the developers, and coordinate static API keys.

With Models-as-a-Service, OpenShift AI handles all of this automatically through a self-service catalog.

Exercise 1: Verify the backend MaaS registration

In the previous section, we explicitly deployed a `MaaSModelRef` custom resource alongside our model. This resource references inference servers served through OpenShift AI and registers them with the MaaS Gateway.

- This is performed by selecting "Publish as MaaS" checkpoint during the deployment configuration process.

Verify the model was successfully registered with the MaaS API:

```
oc get maasmodelref -n chocotech-models
```

Expected output:

NAME	MODELREF_KIND	MODELREF_NAME	READY	AGE
llama-3-8b	LLMInferenceService	llama-3-8b	True	5m

By itself, this `MaaSModelRef` does not grant anyone access. However, because we "pre-wired" governance in Module 2 by explicitly adding `llama-3-8b` to our Gold, Silver, and Bronze Subscriptions and AuthPolicies, the model is now instantly accessible and fully governed by Kuadrant rate limits.

Exercise 2: The Developer Payoff (Generating API Keys)

Let's switch hats. You are now a developer on the Quality Assurance team logging in to find a model for your analysis application. You do not need to know anything about `vLLM` or OpenShift routes; you simply use the self-service catalog.

Creating a Persistent API Key

For a real application, developers need a persistent API key rather than a 1-hour temporary key.

1. In the OpenShift AI dashboard, click **Gen AI studio** → **API keys**.
2. Click **Create API key**.
3. Give the key a descriptive name (e.g., `quality-control-app`).
4. From the **Subscription** dropdown, select the subscription that determines your token limits.
5. Leave the Expiration at the default (30 days) and click **Create**.

Copy the generated key immediately (it starts with `sk-oai-`). The plaintext key is displayed only during creation and cannot be retrieved later.



API Key Snapshots: API keys capture the user's group memberships at the exact time of creation. If a user is removed from an OpenShift group later, their existing API keys retain the original group associations and continue to work until they are explicitly revoked or expired.

Exercise 3: Test API discovery (Listing Models)

The MaaS Gateway is fully compliant with the OpenAI API specification. However, in OpenShift AI 3.4, API discovery is highly secure and strictly scoped. When an application queries the `/maas-api/v1/models` endpoint using a generated `sk-oai-` API key, the Gateway automatically filters the response to show **only** the models allowed by the subscription bound to that specific key.

Retrieve your Gateway URL and query the models list using the persistent API key you just generated:

```
# Ensure your variables are set
export MAAS_API_KEY="sk-oai-..." # Paste your actual key here
export ROUTE_HOST=$(oc get route openshift-ai-inference -n openshift-ingress -o
jsonpath='{.spec.host}')

# Query the MaaS API for available models
curl -H "Authorization: Bearer $MAAS_API_KEY" \
https://{ROUTE_HOST}/maas-api/v1/models | jq
```

You should receive a JSON response listing `llama-3-8b`, confirming that Kuadrant validated your API key, verified your subscription, and successfully authorized the request.

In the next section, we will deploy a second model (Qwen3-4b-code), which will only be available to the Silver and Bronze subscriptions.

3.4: Lab - Publishing to MaaS Catalog

ChocoTech's Marketing and R&D teams need a dedicated coding assistant model (qwen3-4b). This exercise demonstrates how to expand the catalog using the OpenShift AI Dashboard UI, handle limited GPU capacity, and enforce selective access controls.

1. Capacity Management: Scaling down Llama-3

If your lab environment only has a single GPU with limited VRAM (e.g., 24GB), you cannot run both Llama-3 and Qwen simultaneously. As a Platform Engineer, you must free up VRAM before deploying the new model. Scale the Llama-3 deployment to 0 replicas:

```
oc scale llminferenceservice llama-3-8b -n chocotech-models --replicas=0
```

2. Deploy Qwen3 via the Dashboard UI

While GitOps is powerful, OpenShift AI provides a seamless UI for model deployment. First, create the Data Connection secret so the UI knows where the Modelcar image lives:

```
cat <<EOF | oc apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: qwen3-4b
  namespace: chocotech-models
  labels:
    opendatahub.io/dashboard: "true"
  annotations:
    opendatahub.io/connection-type-protocol: uri
    opendatahub.io/connection-type-ref: uri-v1
    openshift.io/display-name: "Qwen 3 8b ModelCar (v1.0)"
stringData:
  URI: oci://quay.io/redhat-ai-services/modelcar-catalog:qwen3-4b
type: Opaque
EOF
```

Now that the data connection is staged, follow these step-by-step instructions to configure the model runtime, hardware allocation, and MaaS governance properties.

1. In the OpenShift AI dashboard, navigate to your `chocotech-models` project and click the **Deployments** tab.
2. Click **Deploy model**.

Step 1: Model Details and Location

Instead of manually typing out the OCI registry URL, we will leverage the internal data connection you just created.

1. **Model location:** Select the down arrow button, then **Existing connection**.
2. **Name:** Select **Qwen 3 4b ModelCar (v1.0)** from the dropdown list.
3. **Folder path:** Leave this blank.
4. **Model Type:** Select **Generative AI model (Example, LLM)**.
5. Click **Next**.

Step 2: Model Deployment Configuration

1. **Name:** Enter `qwen3-4b`.
2. **Hardware profile:** Select **nvidia-gpu** (or your designated gpu-profile).
3. **Deployment resource:** Select **manual**.
4. **Serving runtime:** From the dropdown menu, select **Distributed inference with llm-d**.
5. **Number of replicas:** Leave this set to `1`. (In a production environment, you would scale this for high availability and usage requirements).
6. Click **Next**.

Step 3: Advanced Settings & Governance

This step is critical for integrating the model into ChocoTech's financial and security framework.

1. **Model Availability:** Check the box next to **Publish as MaaS**.

Checking **Publish as MaaS** automatically instructs OpenShift AI to generate a `MaaSModelRef` resource under the hood. This links the newly created model endpoint directly to the MaaS gateway, enabling our Authorino security policies and Kuadrant token quotas to intercept traffic.

2. **Configuration Parameters:** Select **Add custom runtime arguments**. This allows us to fine-tune the vLLM deployment. Add the following parameters:

```
--dtype=half
--max-model-len=8192
--gpu-memory-utilization=0.95
```

(Note: We removed the Llama-3 specific tool-calling parsers from the previous section since we are deploying Qwen).

1. **Routing Settings:** Leave the Rolling update selection and model route timeout selections set to their defaults.
2. Review your configuration choices on the summary pane and click **Deploy model**.

3. The Identity Swap (Testing Selective Access)

Currently, the Qwen model is deployed, but no one has access to it. We want to restrict this model strictly to the **Silver (R&D)** and **Bronze (Marketing)** subscriptions.

1. **Append the model to the Silver and Bronze** `MaaSSubscription` **resources** to grant a token quota:

```

cat <<EOF | oc apply -f -
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSSubscription
metadata:
  name: silver-plan
  namespace: models-as-a-service
spec:
  priority: 50
  owner:
    groups:
      - name: chocotech-rd
    users: []
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
      tokenRateLimits:
        - limit: 10000
          window: 1m
    # NEWLY ADDED MODEL QUOTA
    - name: qwen3-4b
      namespace: chocotech-models
      tokenRateLimits:
        - limit: 2000
          window: 1m
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSSubscription
metadata:
  name: bronze-plan
  namespace: models-as-a-service
spec:
  priority: 10
  owner:
    groups:
      - name: chocotech-marketing
    users: []
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
      tokenRateLimits:
        - limit: 2000
          window: 1m
    # NEWLY ADDED MODEL QUOTA
    - name: qwen3-4b
      namespace: chocotech-models
      tokenRateLimits:
        - limit: 1000
          window: 1m
EOF

```

2. **Append the model to the Silver and Bronze MaaSAuthPolicy resources** to explicitly authorize traffic through the API Gateway:

```

cat <<EOF | oc apply -f -
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: silver-access
  namespace: models-as-a-service
spec:
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
    # NEWLY ADDED MODEL ACCESS
    - name: qwen3-4b
      namespace: chocotech-models
  subjects:
    groups:
      - name: chocotech-rd
    users: []
---
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: bronze-access
  namespace: models-as-a-service
spec:
  modelRefs:
    - name: llama-3-8b
      namespace: chocotech-models
    # NEWLY ADDED MODEL ACCESS
    - name: qwen3-4b
      namespace: chocotech-models
  subjects:
    groups:
      - name: chocotech-marketing
    users: []
EOF

```

The Gold Plan (Quality Control) remains completely untouched. Kuadrant will physically block any attempts from the Gold group to route traffic to the Qwen model.

The Role Swap Challenge: You are currently an administrator in the Gold (Quality Control) group. Let's simulate a department transfer to prove our governance works. Remove yourself from the Gold group and add yourself to the Bronze (Marketing) group:

```

oc adm groups remove-users chocotech-quality-control $(oc whoami)
oc adm groups add-users chocotech-marketing $(oc whoami)

```

4. Revocation and Verification

Because API keys capture a snapshot of user group memberships from when the key was created, your old Gold API key is a security risk. To immediately revoke access after a group change, revoke the API

keys for that user.

1. In the OpenShift AI dashboard, go to **Gen AI studio** → **API keys** and **Revoke** your previous key.
2. Create a new key and select the **Bronze Plan**.
3. Navigate to **Gen AI studio** → **AI asset endpoints**.
4. You will now see `qwen3-4b` listed. Because you are no longer in the Gold group, you are experiencing the exact, filtered catalog view that a Marketing developer sees!

Exercise 3: Verify the multi-model catalog

1. Generate a Test API Key

To accurately test selective access, you must generate an API key tied specifically to the Bronze Plan (Marketing). * In the OpenShift AI dashboard, click **Gen AI studio** → **API keys**. * Click **Create API key**. * Name it `marketing-qwen-test`. * Select **Bronze** from the Subscription dropdown. * Click **Create** and immediately copy the `sk-oai-...` key.

2. The UI Payoff

Refresh the Gen AI Studio Models tab in the OpenShift AI dashboard. * Since you are testing with Bronze/Silver access, you will now see both `llama-3-8b` and `qwen3-4b` listed with their respective subscription access controls. * If you switch to a user in the **Quality Control (Gold)** group, the Qwen model is entirely hidden from the catalog.

3. API Discovery (The Developer View)

Retrieve your Gateway hostname and query the `/maas-api/v1/models` endpoint using the Bronze API Key you just generated:

```
# Ensure variables are set
export MAAS_API_KEY="sk-oai-..." # Paste your Bronze key here
export ROUTE_HOST=$(oc get route openshift-ai-inference -n openshift-ingress -o
jsonpath='{.spec.host}')

curl -s -k -H "Authorization: Bearer $MAAS_API_KEY" \
  https://{ROUTE_HOST}/maas-api/v1/models | jq
```

Expected output showing both models:

```
{
  "object": "list",
  "data": [
    {
      "id": "llama-3-8b",
      "object": "model",
      "owned_by": "chocotech-models"
    },
    {
      "id": "qwen3-4b",
      "object": "model",
      "owned_by": "chocotech-models"
    }
  ]
}
```

(Note: If you run this exact same API call using a Gold Plan API Key, the Gateway will dynamically filter the JSON response to only return `llama-3-8b`).

ChocoTech now operates a fully governed, multi-model AI factory. Quality Control relies on the universally available Llama-3 for factory logic, while Marketing and R&D can utilize the specialized Qwen coder model without risking exposure to the Gold tier production environment.

3.5: Lifecycle Management with GitOps



This section is under development

ChocoTech's Quality Control team has been successfully using the `llama-3-8b` model for several weeks. Now, they want to upgrade to a new version that has been fine-tuned specifically on ChocoTech manufacturing defects.

"We cannot risk downtime," explains the QC Director. "Our production line depends on real-time defect detection. How do we update the model without causing an outage?"

This is a classic "Day 2" production challenge. Because you deployed the model using Models-as-a-Service (MaaS) and GitOps, updating it is safe, auditable, and completely seamless for the end users.

1. Version Management Strategy

Production AI systems require disciplined version management, just like application code. ChocoTech uses semantic versioning for its model weights stored in the OCI registry or local vault:

- **v1.3.0:** Base Llama-3 8B Instruct (Currently deployed)
- **v1.4.0:** Fine-tuned on chocolate defect images (The requested upgrade)
- **v2.0.0:** Major architecture change (e.g., swapping to Llama-3 70B)

By keeping the model weights in distinct versioned paths, you ensure that the previous version is always preserved and ready for an instant rollback.

2. The MaaS Architectural Advantage

In a legacy, ungoverned AI environment, changing a model often meant changing the endpoint URL, issuing new credentials, and forcing developers to update their application code.

OpenShift AI 3.4 eliminates this friction. The Models-as-a-Service platform acts as a governance layer between users and the underlying model serving infrastructure. Because the architecture separates the engine (`LLMInferenceService`) from the registration (`MaaSModelRef`), updating a model's weights is completely decoupled from its access control.

When you update the model weights, the Gold Plan token limits remain perfectly intact, the API keys generated by the developers remain valid, and the Gateway URL does not change.

3. The GitOps Update Workflow

To update the model, a Platform Engineer does **not** delete the running pod. Instead, they update the declarative YAML file in their Git repository.

When updating the `LLMInferenceService`, the only line that needs to change is the `uri` pointing to the new vault location:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: llama-3-8b
  namespace: chocotech-models
# ... [truncating for brevity] ...
spec:
  model:
    uri: oci://quay.io/redhat-ai-services/modelcar-catalog:llama-3-8b-v1.4.0 # <--- Update
    the path
```

Graceful Rollouts vs. The "Pending" Trap

When you commit this change and apply it to the cluster, OpenShift AI and KServe attempt to perform a **Rolling Update**.

1. The system spins up a completely new inference pod and begins downloading the `v1.4.0` weights.
2. During this time, the `v1.3.0` pod continues serving live traffic from the Gateway. No user requests are dropped.
3. Once the `v1.4.0` pod reports a `Ready` status, KServe automatically flips the network route.
4. The old `v1.3.0` pod is then gracefully terminated.



Hardware Troubleshooting: The "Pending Pod" Trap To execute a graceful rollout, your OpenShift cluster **must have an additional equivalent GPU available**.

If you attempt this rolling update in a single-GPU lab environment, your new `v1.4.0` pod will hang in a `Pending` state forever because the `v1.3.0` pod is holding the only GPU hostage! If you encounter this in production, you must either add node capacity or force the update by scaling the old deployment down to 0 first (causing brief downtime).

The Rollback

If the Quality Control team reports that the `v1.4.0` model is hallucinating or underperforming, the rollback process takes seconds. You simply revert the Git commit, and KServe repeats the rolling update process to restore the known-good state.

4. Day-2 Governance and Observability

As ChocoTech's AI usage matures, platform engineers must also manage the lifecycle of user access and platform costs.

- **API Key Lifecycle Management:** You must mandate that teams rotate API keys periodically and revoke compromised keys immediately. As we proved in the previous lab, API keys capture a user's group memberships at the exact time of creation. If a user leaves the Quality Control team, administrators must manually revoke the API keys to immediately revoke access.
- **Cost Attribution (Showback):** As usage scales, Finance will want to know which departments are consuming the most GPU compute. You can use the MaaS observability dashboard to monitor subscription-level usage metrics—such as token consumption and rate-limit violations—and export this usage data in CSV format for cost attribution and showback reporting.

Module 3 Progress Check

You have successfully deployed a multi-model AI factory, secured it with subscription-based governance, and learned how to safely manage the lifecycle of those models and keys in production. You have completed the Platform Engineering infrastructure build! Proceed to the Module 3 Summary.

3.6: Summary and troubleshooting



This section is under development

You have successfully brought the ChocoTech AI Factory online. By deploying these models into the Models-as-a-Service (MaaS) architecture, you have transformed a raw, ungoverned infrastructure into a highly secure, multi-tenant enterprise platform.

If ChocoTech had allowed teams to deploy standard standalone models, they would currently have zero internal cost control and no way to prioritize mission-critical workloads over experimental scripts. By

utilizing the `LLMInferenceService` architecture and explicit `MaaSModelRef` registration, you have secured the company's hardware ROI and established absolute Quality of Service (QoS).

What you accomplished

- **Hardware & Architecture Selection:** Assessed Dense vs. Sparse models and calculated VRAM requirements to ensure ChocoTech's workloads fit efficiently onto their NVIDIA hardware.
- **Secure GitOps Deployment:** Bypassed inefficient public internet downloads by utilizing the OCI Modelcar format, and deployed `llama-3-8b` and `qwen3-4b` using the governed `LLMInferenceService` architecture.
- **MaaS Registration:** Successfully registered the inference engines with the API Gateway by deploying `MaaSModelRef` custom resources.
- **Selective Access Control:** Manually appended the new models to your existing `MaaSSubscription` and `MaaSAuthPolicy` resources to explicitly restrict the Qwen model to Silver and Bronze users, while keeping Gold production workloads isolated.
- **Day-2 Operations:** Mastered the zero-downtime rolling update workflow, allowing you to hot-swap model versions by simply updating the storage URI in your Git repository.

Knowledge check

Test your understanding of the MaaS deployment architecture before moving on:

1. Why must governed models be deployed using `LLMInferenceService` rather than the legacy deployment method?

Models deployed for MaaS use the `LLMInferenceService` architecture, which is designed for large language models and integrates with the MaaS gateway for subscription-based quota enforcement. The legacy deployment method uses traditional `KServe InferenceService` resources with serving runtimes, which bypass the MaaS architecture entirely.

2. What happens if a model is deployed and registered as a `MaaSModelRef`, but the user's group is not added to a `MaaSAuthPolicy`?

The model will exist on the cluster, but no user will be able to route traffic to it. Without an authorization policy, users receive `403 Forbidden` errors even if they have a valid subscription.

3. How does the Gateway API discovery endpoint (`/maas-api/v1/models`) handle model visibility when a developer authenticates with an `sk-oai-` API key?

The API gateway intercepts the request and validates the provided API key. Because the key is permanently bound to a specific subscription, the endpoint dynamically filters the JSON response to show **only** the models allowed by that exact subscription.

Day-2 Troubleshooting: Model Deployment & Catalog

As a Platform Engineer, you will field support tickets when models fail to deploy or users cannot connect. Use this guide to resolve common Day-2 issues:

Issue: Model Pod stuck in `Pending` during an update

The Hardware Trap: You likely attempted a rolling update without spare GPU capacity. KServe keeps the old pod running (consuming the GPU) while trying to start the new pod.

- **The Fix:** If you lack spare GPUs, you must scale the old `llminferenceservice` down to 0 replicas to free the GPU, then scale it back up with the new configuration.

Issue: The model does not appear in the Gen AI Studio catalog

- **Verify Registration:** Check that the `MaaSModelRef` was created using `oc get maasmodelref -n <your-project-namespace>`. If the object does not exist, the operator did not register the model.
- **Use legacy deployment method:** If deployed via the UI, ensure the **Use legacy deployment method** checkbox was unchecked. Legacy deployments bypass the MaaS architecture entirely.

Issue: User access errors (403 Forbidden)

- **Missing Policies:** Verify that the user has both a subscription and an authorization policy. A subscription grants quota for specific models with token limits. An authorization policy is required to authorize groups to access model endpoints through the API gateway.
- **Selective Access Mismatch:** Confirm the user belongs to the OpenShift group explicitly listed in the AuthPolicy subjects.

Issue: Authentication errors (401 Unauthorized)

- **Invalid Tokens:** Ensure you are using a MaaS API key generated from the dashboard. OpenShift tokens are not valid for MaaS authentication.
- **Revoked/Expired Keys:** The API key might be expired or manually revoked. Generate a new MaaS API key.

Issue: Model not found (404)

- **Typo in URL:** Ensure the URL format is correct: `https://maas.<domain>/llm/<model-name>/v1/chat/completions`.
- **Incorrect Model Name:** Verify the exact model name using the `/v1/models` discovery endpoint.

Your journey progress

✓ Module 1

Architecture
Foundation

✓ Module 2

& Policies & Governance

✓ Module 3

Model Deployment

🕒 Module 4

Application Integration

“**Next steps:** The AI Factory is fully operational. In Module 4, you will switch hats from Platform Engineer to Application Developer. You will use Python, LangChain, and your new API keys to connect ChocoTech’s applications directly to the MaaS Gateway, and write the exponential backoff logic required to handle Kuadrant’s rate limits!

4: Observability



This section is under development

ChocoTech's AI Factory is live, and the business units are successfully accessing the multi-model catalog. However, on day three of production, you receive an urgent Slack message from the lead developer on the Marketing team (Bronze Subscription):

“Our new batch-processing script for sentiment analysis keeps crashing with a '429 Too Many Requests' error. Also, Finance just emailed asking how much of the monthly cloud budget our department consumed this week. How do we fix the script, and where do I get the billing data?”

This module bridges the final gap between the Application Developer and the Platform Engineer. You will step into the developer's shoes to write a robust application that gracefully handles Models-as-a-Service (MaaS) rate limits. Then, you will switch back to the platform engineer role to enable telemetry, analyze the resulting metrics, and export the automated chargeback reports for Finance.



The Silent Telemetry Trap

OpenShift AI will report a 100% healthy MaaS installation even if User Workload Monitoring (UWM) is disabled on the OpenShift cluster. The Gateway and Kuadrant rate-limiting components do not require Prometheus to enforce rules. However, if UWM is not enabled, your platform will silently drop all token tracking data, leaving your chargeback dashboards completely blank! You will verify this pipeline in this module.

Learning objectives

By the end of this module, you will be able to:

- Implement exponential backoff in client applications to gracefully handle Kuadrant's 429 Too Many Requests rate limits.
- Configure Kuadrant and Tenant custom resources to enable platform-wide rate-limiting metrics and token telemetry.
- Navigate the MaaS observability dashboard to track token consumption by model and subscription.
- Export token-level usage data in CSV format for automated financial chargeback workflows.

The execution strategy

To solve the Marketing team's errors and satisfy Finance's billing requirements, you will execute the following strategy:

Phase		Action
1. Application Integration (The Chaos Generator)		You will run a Python load-generator script using the Marketing team's API key. This script will intentionally attempt to breach the Bronze tier's token quota to demonstrate how developers must handle HTTP 429 rate limits in their code.
2. Enabling Telemetry		The load generator will create a massive spike in traffic. You will patch the Kuadrant and Tenant custom resources to instruct Prometheus to scrape the token consumption metrics.
3. The Dashboard	Observability	You will access the MaaS observability dashboard to visualize the chaos you just created. You will analyze Total Tokens, Request Counts, and the specific rate-limit violations triggered by the Python script.
4. Cost (Chargeback)	Attribution	Finally, you will export the usage data into a structured CSV format, providing the Finance team with the exact mathematical proof they need to bill the Marketing department.

By the end of this module, ChocoTech will have a fully monitored, financially accountable, and developer-friendly AI ecosystem.